chart를 front-end에서 구현 메모

답변에 그래프(등의 추가 데이터) 추가용 메모

BE에서 그려서 넘겨주는 방식에서 BE에선 데이터만, FE에서 chart 구현으로 변경

시뮬레이션의 경우 데이터가 BE에서 존재하는지라 전처리 후 넘겨주고 있지만 경우에 따라서 데이터를 불러오는 방식은 바뀔 수 있지 않을까...

전처리

시뮬레이션 데이터 10000개를 그냥 보내기엔 너무 의미가 없어 bins 기반으로 쪼갠 데이터를 전송하는 방식으로 전처리

```
    def _generate_chart(self, results: list, avg: float, p20: float) -> dict:
        min_val, max_val = min(results), max(results)
        bin_count = 40
        bin_size = (max_val - min_val) / bin_count

        bins = []
        for i in range(bin_count):
            left = min_val + i * bin_size
            right = left + bin_size
            count = sum(1 for r in results if left <= r < right)
            bins.append({
                "left":  round(left),
                "right": round(right),
                "count": count,
                "type":  "loss" if left < 0 else "p20" if left < p20 else "profit"
            })

        return {
            "bins":     bins,
            "avg":      round(avg),
            "p20":      round(p20),
            "min":      round(min_val),
            "max":      round(max_val),
        }
```

함수명은 다른데까지 손대기 귀찮아서 이전 BE넘겨주던 함수명 그대로 사용

retern값이 데이터로 변경

사실 함수의 내용은 전처리가 전부라 데이터에 맞춰서 작성 할 부분이겠다...

함수명을 일부러 바꾸지 않아 다른 return은 확인 안했지만... 실제 front로 넘어가는 부분도 확인해주자

답변에 그래프(등의 추가 데이터) 추가용 메모 이외 BE변경점은 이쪽과 동일

```
# 기존 simulation의 return부분에 chart 추가
        chart = self._generate_chart(results, p20, avg) # 입력변수가 바뀌진 않는지 확인해볼법하다
        return {
               # 기존 return 변수들...
            "chart":              chart,
        }
# 이 외 agent 부분, orchestrator.py, api_server.py 도 확인
```

Front-End부분

UserChat.jsx

유저 입력 및 결과 불러오기 파트

chartData를 저장-전달해야하기 때문에 아래와 같이 추가

```
async function handleSubmit(question) {
   // ...
    if (finalResult) {
      setMessages(prev => [
        ...prev,
        {
          question,
          domain:         finalResult.domain,
          status:         finalResult.status,
          grade:          finalResult.grade,
          confidenceNote: finalResult.confidence_note,
          draft:          finalResult.draft,
          retryCount:     finalResult.retry_count,
          chartData:      finalResult.chart || null, // <- 차트 데이터 추가
        },
      ]);
    }

    // return의 메세지 관련 div
        <div className="flex flex-col gap-6">
          {messages.map((msg, i) => (
            <ResponseCard
              key={i}
              question={msg.question}
              domain={msg.domain}
              status={msg.status}
              grade={msg.grade}
              confidenceNote={msg.confidenceNote}
              draft={msg.draft}
              retryCount={msg.retryCount}
              showMeta={false}
              chartData={msg.chartData}
            />
```

```
          ))}
```

ResponseCard.jsx

실제 출력되는 답변 파트

차트 데이터와 차트 import

```
import SimulationChart from "./SimulationChart";
// 실제 차트 구현 파일. 직접 타이핑이면 자동import 될텐데... 일단 변경사항이니
...
export default function ResponseCard({ question, domain, status, grade, confidenceNote, draft, retryCount,
   // 마지막에 chartData 추가

   ...
               <ReactMarkdown>{draft}</ReactMarkdown>
               <SimulationChart chartData={chartData}/>
               // 답변 draft 이후에 chartData 추가
   ...
             </div>
           ) : (
             <>
             <ReactMarkdown>{draft}</ReactMarkdown>
             <SimulationChart chartData={chartData}/>
             </>
             // 기존 draft부분 아래 추가 > 단일 DOM이었던 draft부분이 2개 DOM으로 바뀌는 부분이라 전체를 <></>로 감
           )}
         </div>
```

차트 구현 파일(신규 작성)

SimulationChart. jsx

해당파일에 구현

chart.js 를 사용하기 때문에 설치해준다

```
cd frontend
npm install chart.js
```

우선 전문을 첨부. 아래 세부사항 有

전문

```
import { useEffect, useRef } from "react";
import { Chart } from "chart.js/auto";

export default function SimulationChart({ chartData }) {
    const canvasRef = useRef(null);
    const chartRef = useRef(null);
```

```
  useEffect(() => {
      if (!chartData || !canvasRef.current) return;

      if (chartRef.current) chartRef.current.destroy();

      const labels = chartData.bins.map(b =>
          `${Math.round(b.left / 10000)}만`
      );
      const data = chartData.bins.map(b => b.count);
  const colors = chartData.bins.map(b =>
      b.type === "loss"   ? "#E24B4A" :
      b.type === "p20"    ? "#EF9F27" : "#378ADD"
  );

  chartRef.current = new Chart(canvasRef.current, {
      type: "bar",
    data: {
        labels,
        datasets: [{
            data,
            backgroundColor: colors,
            borderWidth: 0,
            barPercentage: 1.0,
            categoryPercentage: 1.0,
          }]
      },
    options: {
        responsive: true,
      plugins: {
          legend: { display: false },
          tooltip: {
              callbacks: {
                  title: (items) => items[0].label + "원 구간",
                  label: (item) => "빈도: " + item.raw + "회",
              }
          }
      },
      scales: {
          x: { ticks: { maxTicksLimit: 8 }, grid: { display: false } },
          y: { ticks: { maxTicksLimit: 5 } }
      }
    }
  });

  return () => chartRef.current?.destroy();
}, [chartData]);

if (!chartData) return null;
if (!chartData.bins) return null;

return (
```

```
    <div className="mt-4">
      <div className="flex gap-4 text-xs text-slate-400 mb-2">
        <span><span className="inline-block w-2.5 h-2.5 rounded-sm bg-[#E24B4A] mr-1"/>손실</span>
        <span><span className="inline-block w-2.5 h-2.5 rounded-sm bg-[#EF9F27] mr-1"/>하위 20%</span>
        <span><span className="inline-block w-2.5 h-2.5 rounded-sm bg-[#378ADD] mr-1"/>수익</span>
      </div>
      <div style={{ position: "relative", height: "200px" }}>
        <canvas ref={canvasRef} />
      </div>
    </div>
  );
}
```

세부사항

- 고정구조

```
const canvasRef = useRef(null);  // canvas DOM 참조
const chartRef = useRef(null);   // Chart.js 인스턴스 참조

useEffect(() => {
    if (chartRef.current) chartRef.current.destroy();  // 이전 차트 제거 (메모리 누수 방지)
    ...
    return () => chartRef.current?.destroy();  // 언마운트 시 정리
}, [chartData]);

if (!chartData) return null;  // useEffect 아래에 위치해야 함
```

chart.js가 canvas를 조작하는 구조라고 한다.

```
<div style={{ position: "relative", height: "200px" }}>
    <canvas ref={canvasRef} />
</div>
```

div 부분에 height 지정 / canvas에는 style 지정 X

- 데이터에 따라 변경(모르겠으면 AI한테 물어봅시다!)

이 외 label단위 / color / type 같은 부분은 구조에 따라 바꿔주면 될 것 같다

```
    useEffect(() => {
        if (!chartData || !canvasRef.current) return;

        if (chartRef.current) chartRef.current.destroy();

        // label파트
```

```
    const labels = chartData.bins.map(b =>
        `${Math.round(b.left / 10000)}만`
    );
    const data = chartData.bins.map(b => b.count);

// color파트
const colors = chartData.bins.map(b =>
    b.type === "loss"   ? "#E24B4A" :
    b.type === "p20"    ? "#EF9F27" : "#378ADD"
);
```

실제 그래프 파트

```
chartRef.current = new Chart(canvasRef.current, {
    type: "bar", // 그래프 타입(재무의 경우 바-히스토그램타입)
  data: {
      labels,
      datasets: [{
          data,
          backgroundColor: colors,
          borderWidth: 0,
          barPercentage: 1.0,
          categoryPercentage: 1.0,
        }]
    },
  options: {
      responsive: true,
    plugins: {
        legend: { display: false },
        tooltip: {
            callbacks: {
                title: (items) => items[0].label + "원 구간",
                label: (item) => "빈도: " + item.raw + "회",
            }
        }
    },
    scales: {
        x: { ticks: { maxTicksLimit: 8 }, grid: { display: false } },
        y: { ticks: { maxTicksLimit: 5 } }
    }
  }
});
```

return파트

```
<div className="mt-4">
  <div className="flex gap-4 text-xs text-slate-400 mb-2">
    <span><span className="inline-block w-2.5 h-2.5 rounded-sm bg-[#E24B4A] mr-1"/>손실</span>
    <span><span className="inline-block w-2.5 h-2.5 rounded-sm bg-[#EF9F27] mr-1"/>하위 20%</span>
    <span><span className="inline-block w-2.5 h-2.5 rounded-sm bg-[#378ADD] mr-1"/>수익</span>
```

```
        // 이 3개는 범례 표기(차트 위 손실 하위20% 수익 부분)
      </div>
      <div style={{ position: "relative", height: "200px" }}>
        <canvas ref={canvasRef} />
      </div>
    </div>
```

일단 전체 구조-수정사항은 이렇고, 그래프 세부 내용(지금은 클릭해야 보이는데 호버식는 안되는지... 그 내부 내용 수정 좀 할 것도 같고... 이런 자잘한 부분) 수정하러 갈 것 같다... 야호...